

SISTEMAS OPERATIVOS

3004610 - 1

Germán Sánchez Torres, I.S., M.Sc., Ph.D.

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com



Conceptos arquitectónicos del computador

Contenido

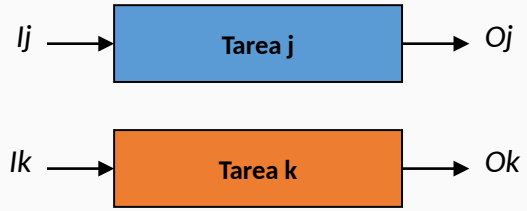
1. Estructura y funcionamiento del computador
2. Modelos de programación
3. Interrupciones
4. Reloj
5. Segmentación y arquitecturas RISC y CISC
6. Procesador, memoria y almacenamiento: tendencias



Estructura y funcionamiento del computador

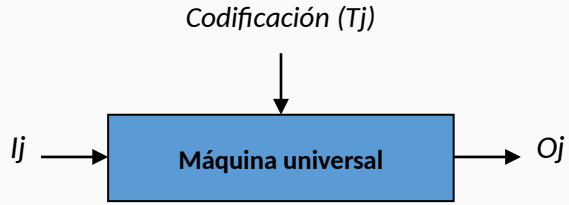
Computación personalizada frente a programa almacenado

Computación digital personalizada



Circuitos diferentes para cada tarea

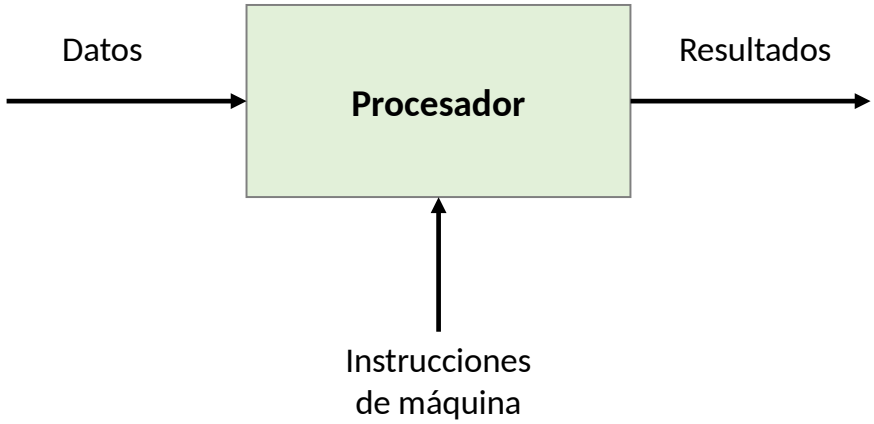
Computación de programas almacenados



Único circuito

- Von Neumann mostró que existe una **máquina universal** (hardware) que puede personalizarse para realizar tareas diferentes mediante entradas de control.
- El **software** es la codificación de la tarea que controla la máquina.
- Hay una **única unidad de almacenamiento**, porque instrucciones y datos no presentan diferencias: el programa se guarda en la misma memoria que los datos.

Estructura general a alto nivel

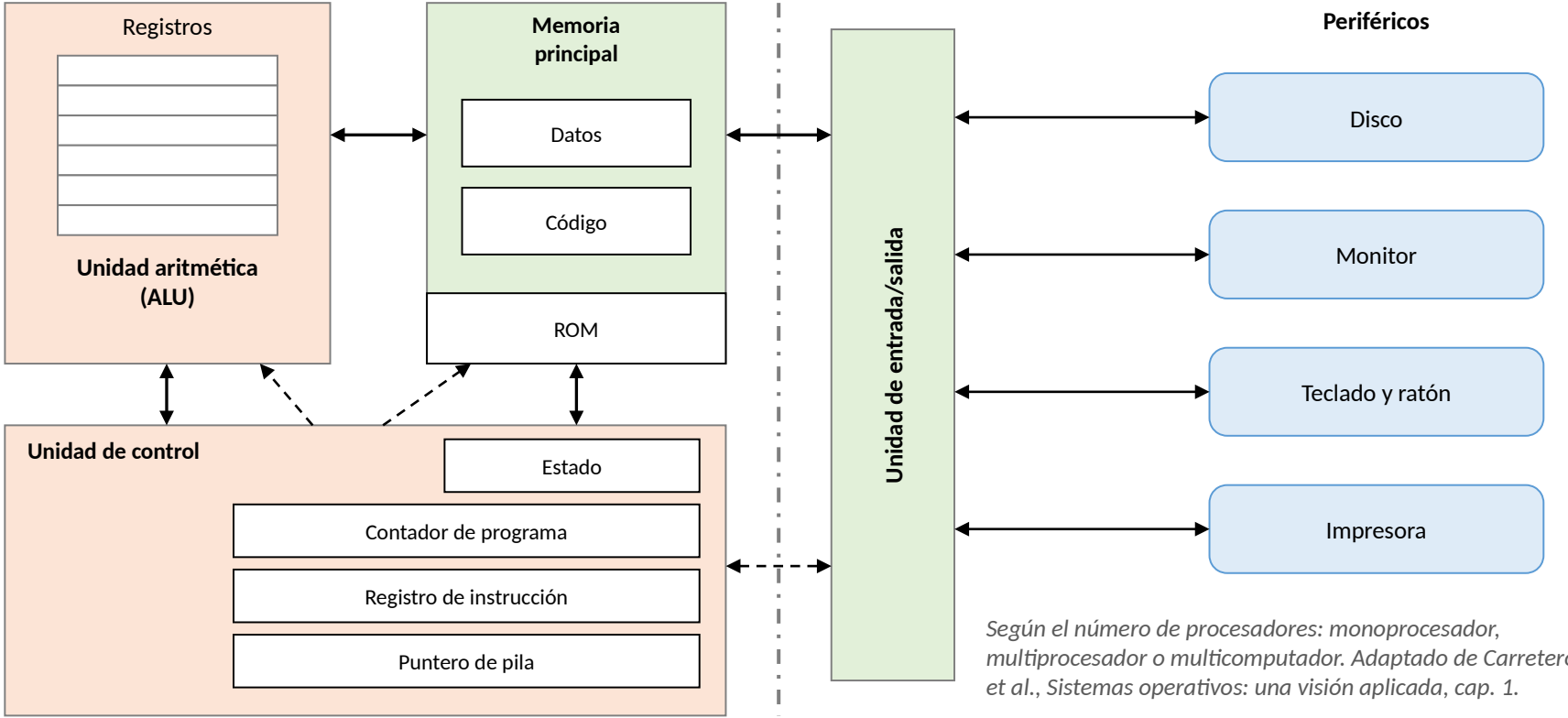


- El procesador transforma los **datos** en **resultados** ejecutando **instrucciones de máquina** almacenadas en memoria.

Funciones de la unidad de control

- Lectura de las instrucciones de máquina.
- Interpretación (decodificación) de las instrucciones.
- Lectura de los datos de memoria.
- Ejecución de las instrucciones.
- Almacenamiento de los resultados.

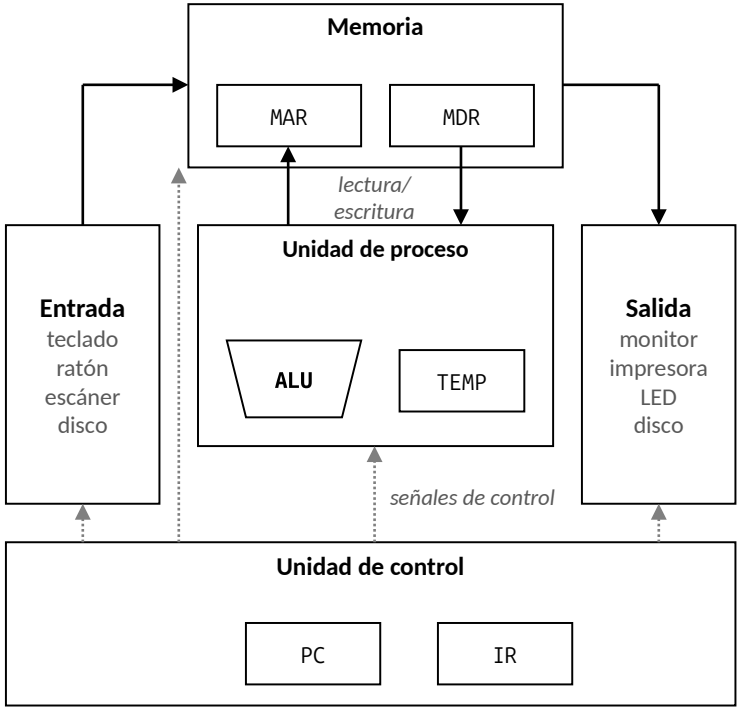
Modelo de Von Neumann



Según el número de procesadores: monoprocesador, multiprocesador o multicomputador. Adaptado de Carretero et al., Sistemas operativos: una visión aplicada, cap. 1.

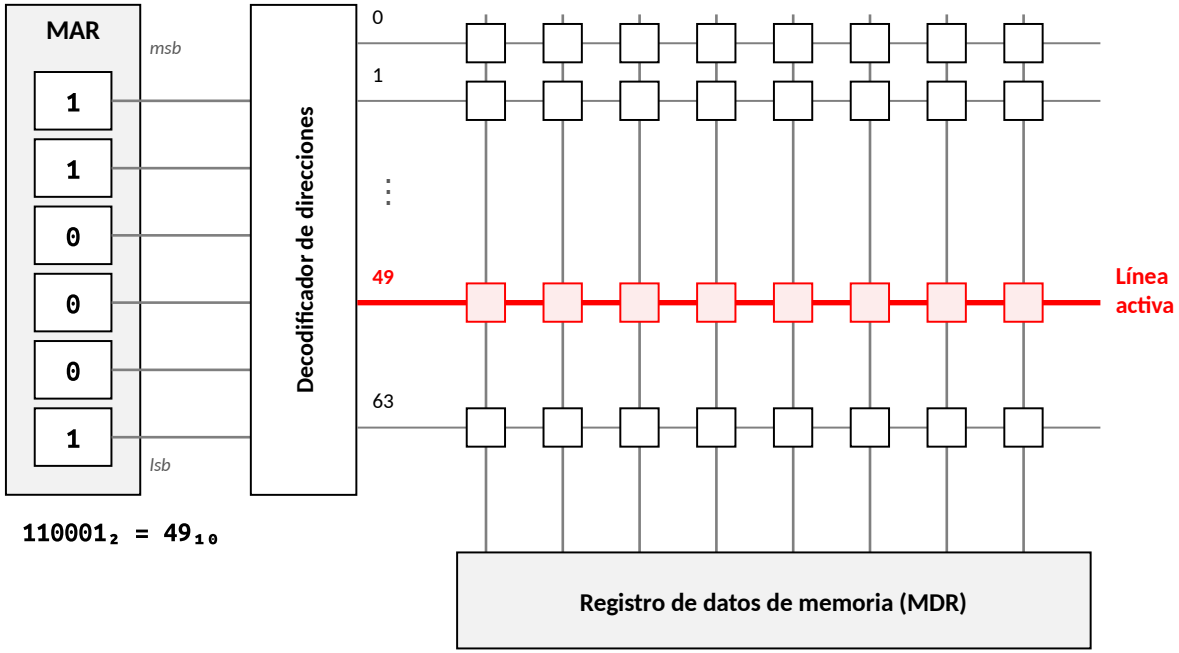
Registros de la unidad de control y de acceso a memoria

Registro	Contenido
Contador de programa (PC)	Dirección de la siguiente instrucción a ejecutar.
Puntero de pila (SP)	Dirección de la cima de la pila.
Registro de instrucción (IR)	Instrucción que se está ejecutando.
Registro de estado (SR)	Información producida por las instrucciones más recientes: bits de estado aritmético, bits de interrupción, nivel de ejecución, etc.
Registro de dirección de memoria (MAR)	Dirección de la palabra de memoria que se va a leer o escribir.
Registro de datos de memoria (MDR)	Palabra leída de la memoria o que se va a escribir en ella.



MAR y MDR conectan la unidad de control con la memoria: la unidad de control indica la operación con la señal de lectura/escritura.

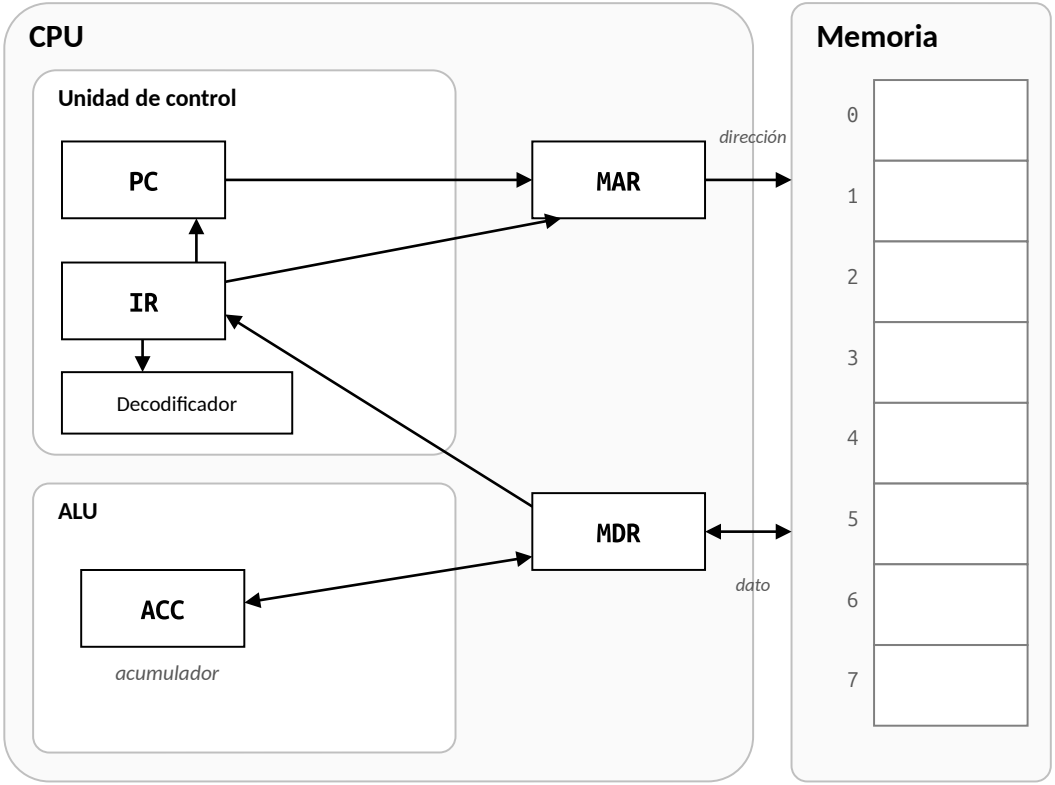
Acceso a una palabra de memoria: MAR, decodificador y MDR



$110001_2 = 49_{10}$

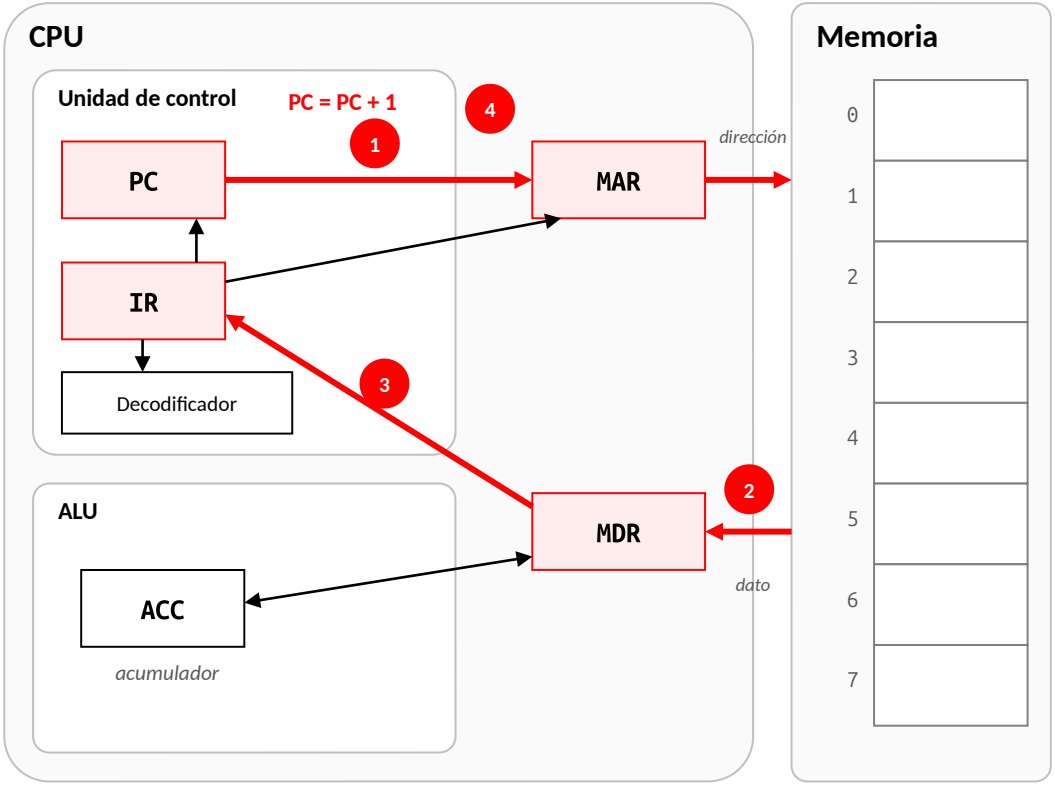
- El **MAR** guarda la dirección de la palabra (con 6 bits, 64 palabras).
- El **decodificador** activa una única línea de palabra: la 49 (110001_2).
- Las celdas de esa línea se conectan a las líneas de bit: su contenido pasa al **MDR** (lectura) o se sobrescribe con él (escritura).
- La unidad de control indica la operación con la señal de lectura/escritura.

El ciclo leer-decodificar-ejecutar: modelo de CPU



- La CPU funciona repitiendo sin fin el ciclo **leer → decodificar → ejecutar**.
- **PC**: dirección de la siguiente instrucción.
- **MAR**: dirección de memoria a la que se accede.
- **MDR**: dato leído de la memoria o que se va a escribir.
- **IR**: registro de instrucción (la que se está ejecutando).
- **ACC**: acumulador, donde la ALU deja sus resultados.
- La unidad de control decodifica el IR y gobierna el flujo entre registros, ALU y memoria.

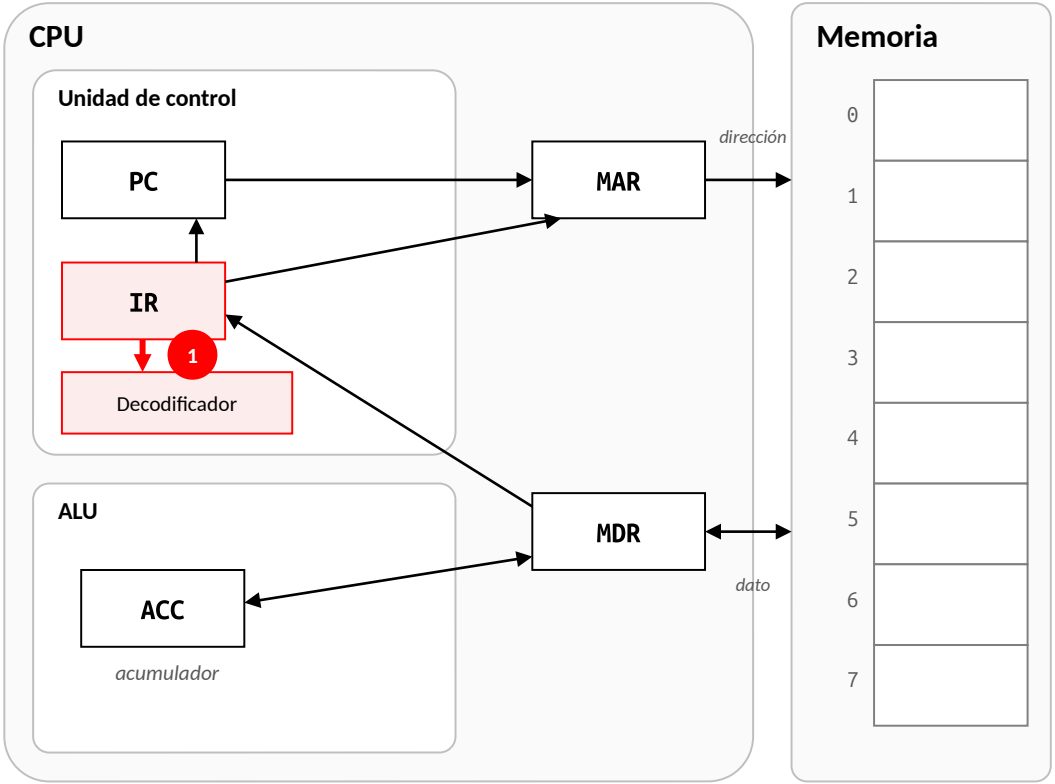
Fase 1: leer la instrucción (fetch)



1. Se copia en el **MAR** la dirección de la siguiente instrucción, contenida en el **PC**.
2. La memoria entrega la instrucción almacenada en esa dirección al **MDR**.
3. La instrucción pasa del **MDR** al registro de instrucción **IR**.
4. El **PC** se incrementa en 1 para apuntar a la siguiente instrucción.

- Esta fase es igual para todas las instrucciones: la CPU todavía no sabe qué va a hacer.

Fase 2: decodificar la instrucción



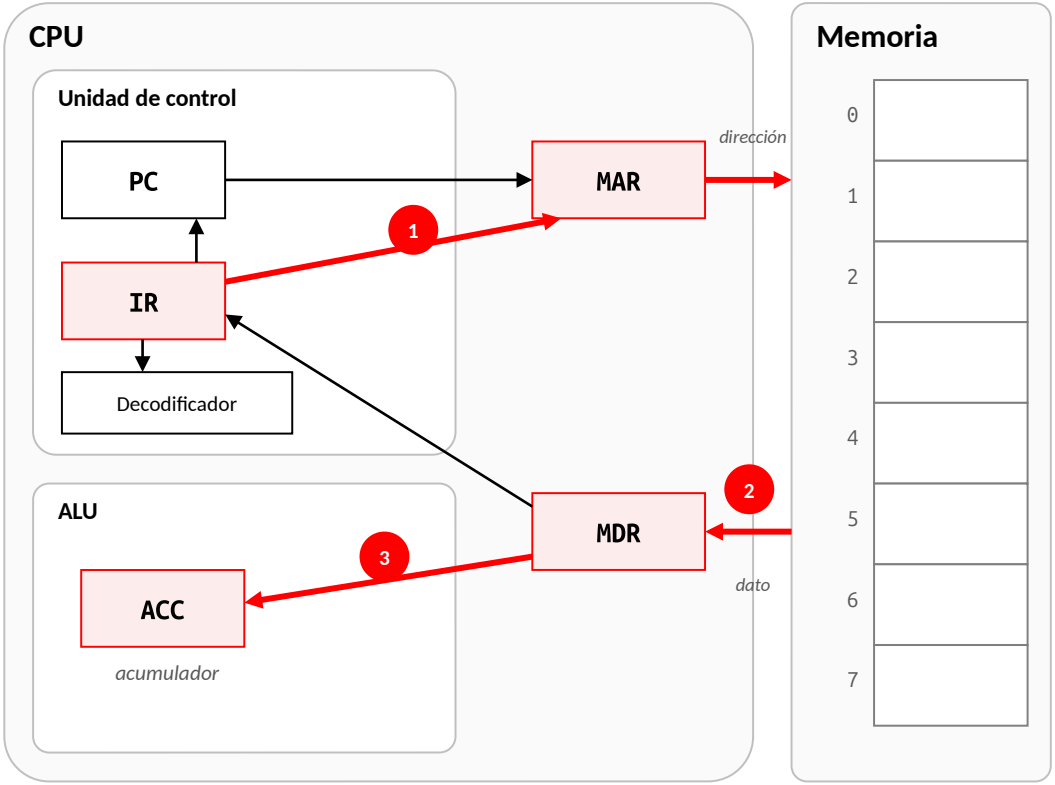
1 La unidad de control separa la instrucción del **IR** en **código de operación** y, si existe, **dirección u operando**.

- Decodifica el código de operación: determina qué hay que hacer y qué señales de control activar.
- Ejemplo de codificación (x86, 8 bits de código + 8 bits de dato):

```
MOV BL, 055H
opcode (mover 8 bits a BL): 10110011
dato 055H:                  01010101
instrucción: 1011001101010101
```

El código de operación (10110011 = B3H: 10110 = MOV inmediato de 8 bits, 011 = registro BL) indica la acción; el operando (01010101 = 055H) es el dato inmediato.

Fase 3: ejecutar la instrucción



- Lo que ocurre depende de la instrucción decodificada. Ejemplo: **Add desde memoria**:

1. La parte de dirección del **IR** se copia en el **MAR**.
2. La memoria entrega el dato de esa dirección al **MDR**.
3. La ALU suma el **MDR** al acumulador: **ACC = ACC + MDR**.

- Un salto (Jump) copia la dirección en el **PC**; un Store envía **ACC** → **MDR** → memoria; un Output envía el **ACC** al dispositivo de salida.
- Al terminar, el ciclo vuelve a la fase 1 con el **PC** ya incrementado.

Juego de instrucciones del ejemplo

Instrucción	Qué hace	Flujo de datos en la ejecución
Jump (salto)	Continúa la ejecución en la dirección indicada.	Dirección del IR → PC
Input / Load (valor directo)	Carga en el acumulador el número contenido en la instrucción.	Operando del IR → ACC
Input / Load (desde memoria)	Carga en el acumulador el dato guardado en una dirección de memoria.	Dirección → MAR; memoria → MDR → ACC
Add (valor directo)	Suma al acumulador el número contenido en la instrucción.	ACC = ACC + operando (ALU)
Add (desde memoria)	Suma al acumulador el dato de una dirección de memoria.	Dirección → MAR; memoria → MDR; ACC = ACC + MDR
Store (almacenar)	Guarda el acumulador en una dirección de memoria.	Dirección → MAR; ACC → MDR → memoria
Output (desde el acumulador)	Envía el contenido del acumulador al dispositivo de salida.	ACC → salida
Output (desde memoria)	Envía a la salida el dato de una dirección de memoria.	Dirección → MAR; memoria → MDR → salida
Reset (reinicio del ciclo)	No es una instrucción: al terminar la ejecución, el control vuelve al PC y comienza una nueva lectura.	PC → siguiente ciclo leer-decodificar-ejecutar

Modelo didáctico de CPU con acumulador: las instrucciones «directas» llevan el operando en la propia instrucción; las «desde memoria» llevan la dirección del dato.

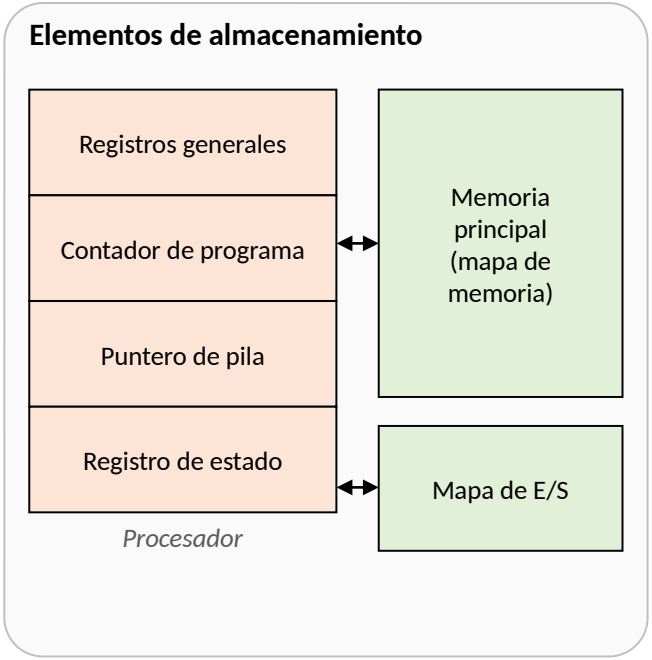


Modelos de programación

El modelo de programación a bajo nivel

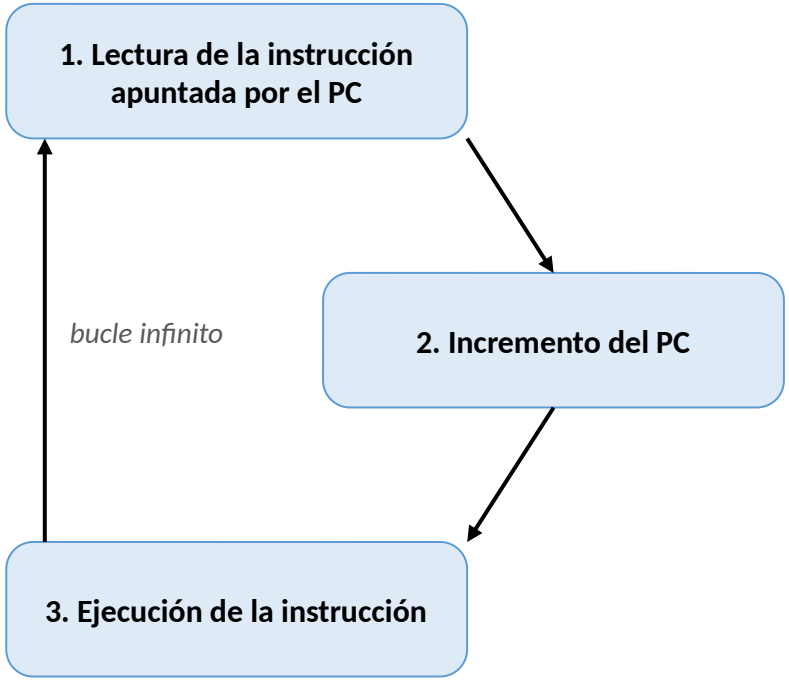
El modelo de programación, a bajo nivel, se basa en:

1. **Elementos de almacenamiento** disponibles en la ejecución de las instrucciones de máquina: registros generales, contador de programa, puntero de pila, registro de estado, memoria principal y mapa de E/S.
2. **Juego de instrucciones**: operaciones que puede realizar el computador, con sus modos de direccionamiento.
3. **Secuencia de funcionamiento**: modo en que se van ejecutando las instrucciones de máquina; una secuencia sencilla que se repite a altísima velocidad en un bucle infinito.
4. **Niveles de ejecución**: nivel de usuario y nivel de núcleo.



Las instrucciones de máquina operan sobre estos elementos.

Secuencia de funcionamiento: el bucle de ejecución

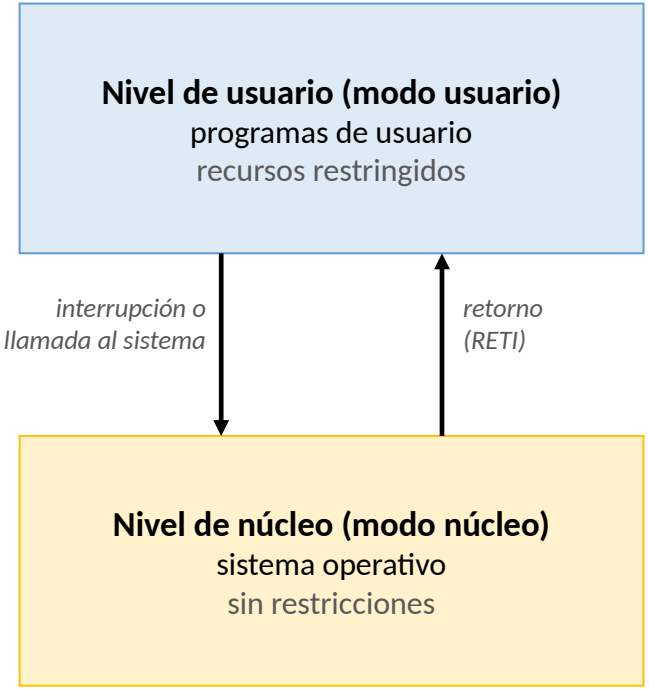



```
mientras (verdadero) {
    IR = memoria[PC];   // 1. lectura
    PC = PC + 1;        // 2. incremento del PC
    ejecutar(IR);       // 3. ejecución
}
```

- **Secuencia lineal**: se ejecutan instrucciones consecutivas, salvo que una instrucción de salto cambie el PC.
- **Bucle infinito**: el procesador nunca deja de ejecutar este ciclo mientras está encendido.
- Esta secuencia es la base sobre la que el sistema operativo construye la ejecución de programas y la atención de interrupciones.

Niveles de ejecución: modo usuario y modo núcleo

- Generalmente los sistemas operativos disponen de **varios niveles de ejecución**.
- **Nivel de usuario** (modo usuario): algunos recursos no están disponibles, para evitar que un programa de un usuario pueda interferir en la ejecución de otros programas o hacer uso de recursos que no le pertenecen (por ejemplo, archivos).
- **Nivel de núcleo** (modo núcleo): no hay restricciones; en él se ejecuta el sistema operativo.
- El nivel actual se guarda en el registro de estado; el paso de usuario a núcleo ocurre en las interrupciones y en las llamadas al sistema (instrucción TRAP).



Modelo de programación de usuario frente al de núcleo

Modelo de programación de usuario

- Registros de datos D0 – D7
- Registros de dirección A0 – A6
- A7: puntero de pila de usuario
- Contador de programa (PC)
- Registro de estado: solo el octeto de usuario (X N Z V C)
- Mapa de memoria: 0 … $2^{31} - 1$
- Juego de instrucciones: sin las instrucciones privilegiadas

T: modo traza · S: sistema/usuario · X N Z V C: extensión, negativo, cero, desbordamiento, acarreo.

Modelo de programación de núcleo

- Registros de datos D0 – D7
- Registros de dirección A0 – A6
- A7: puntero de pila de usuario y A7': puntero de pila de sistema
- Contador de programa (PC)
- Registro de estado completo: octeto de sistema (T, S, máscara de interrupciones) y de usuario
- Mapa de memoria completo: 0 … $2^{32} - 1$
- Mapa de E/S: 0 … $2^{16} - 1$
- Juego de instrucciones completo

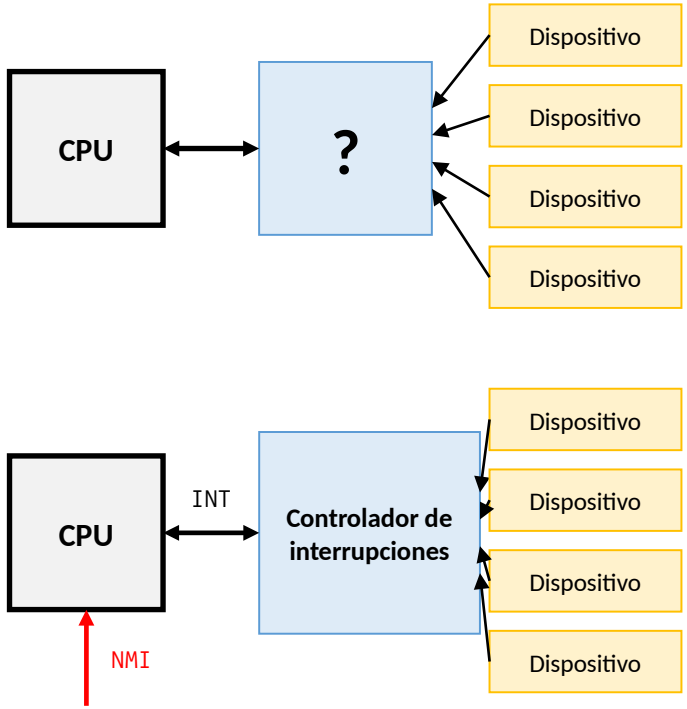
En rojo, lo que solo está disponible en el nivel de núcleo. Adaptado de Carretero et al., Sistemas operativos: una visión aplicada, cap. 1 (registros del Motorola 68000).



Interrupciones

¿Cómo interrumpen los dispositivos al procesador?

- ¿Pueden varios dispositivos interrumpir al procesador?
  - ¿Cómo reconoce el procesador qué dispositivo provoca la interrupción?
  - ¿Cómo sabe qué rutina de tratamiento de la interrupción (ISR) debe ejecutar?
  - ¿Puede una interrupción interrumpir a una ISR?
  - ¿Cómo se atienden solicitudes simultáneas?
- **Solución simple: sondeo** (polling)
  - Consultar cada dispositivo capaz de interrumpir y comprobar si su bit de estado indica que hay algo que hacer; entonces se atiende.
  - Ventaja: simple. Desventaja: lento.
- **Solución habitual: un controlador de interrupciones** que recibe las solicitudes, las prioriza e identifica al dispositivo. Además, el procesador tiene una línea **no enmascarable** (NMI) para eventos graves.



Qué es una interrupción y ciclo de aceptación

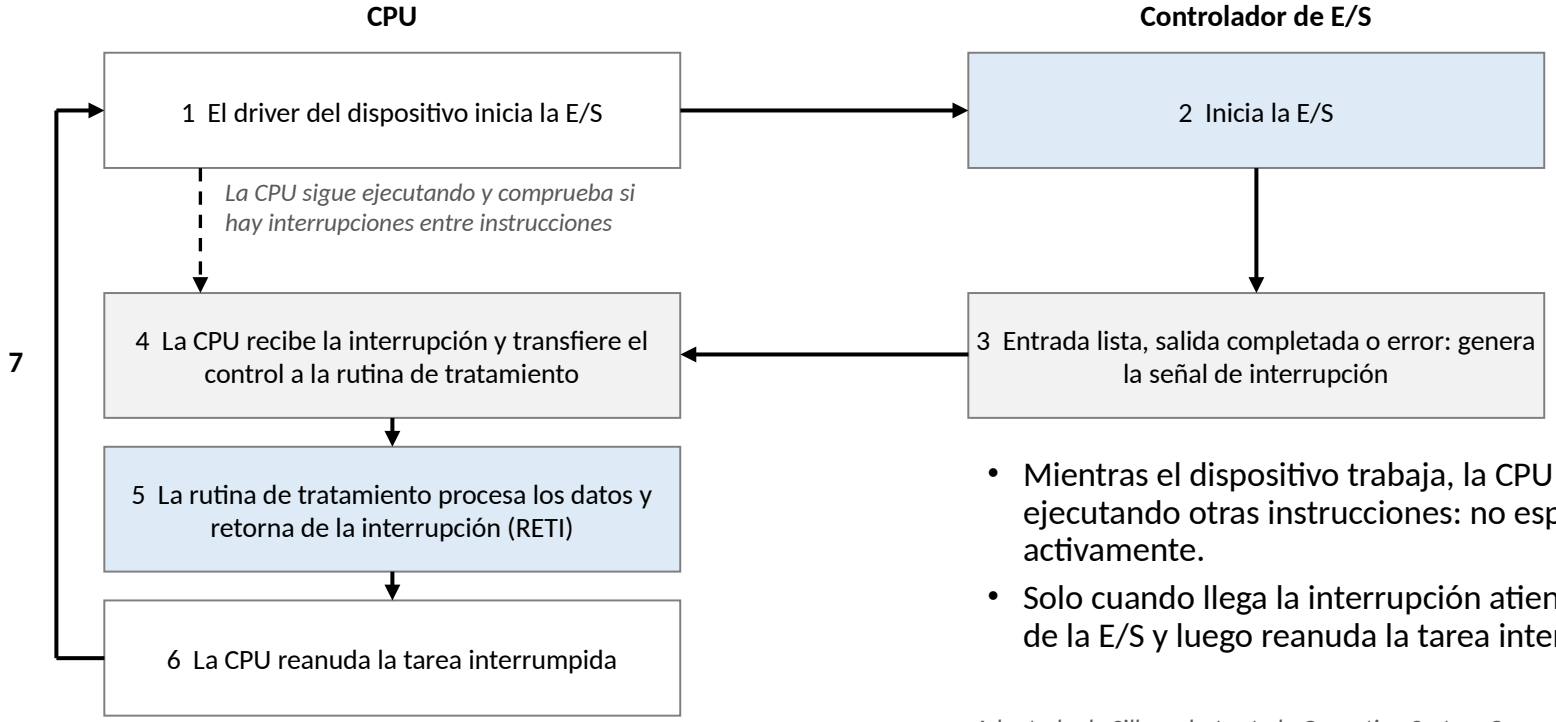
- Una interrupción se solicita activando una **señal** que llega a la unidad de control. El elemento que necesita ser atendido y genera la señal se denomina **agente generador**.
- Ante la solicitud (si la interrupción está habilitada), el procesador realiza un **ciclo de aceptación** al terminar la instrucción en curso:
  1. Guarda algunos registros del procesador (PC y registro de estado).
  2. Eleva el nivel de ejecución del procesador (nivel de núcleo).
  3. Salta al sistema operativo cargando un nuevo valor en el PC.
- El sistema operativo dispone de **rutinas de tratamiento** de las interrupciones.

Bucle de ejecución con atención de interrupciones

```
mientras (verdadero) {
    IR = memoria[PC];
    PC = PC + 1;
    ejecutar(IR);
    si (hay interrupciones pendientes) {
        // atender la interrupción
        guardar PC y registro de estado;
        elevar el nivel de ejecución;
        PC = dirección de la rutina;
    }
}
```

La comprobación se hace **entre instrucciones**: la instrucción en curso siempre termina antes de atender la interrupción. En rojo, el ciclo de aceptación; la rutina de tratamiento es parte del sistema operativo.

Ciclo de E/S regido por interrupciones



- Mientras el dispositivo trabaja, la CPU sigue ejecutando otras instrucciones: no espera activamente.
- Solo cuando llega la interrupción atiende el resultado de la E/S y luego reanuda la tarea interrumpida (7).

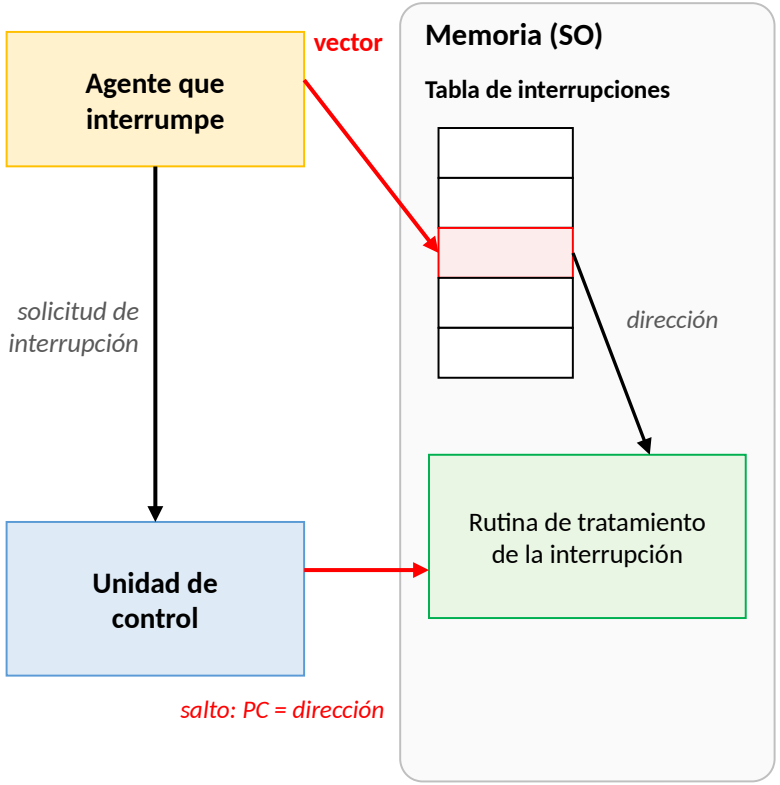
Adaptado de Silberschatz et al., Operating System Concepts, 10.ª ed., cap. 1.

Mecanismo de vector de interrupción

- Mecanismo usual para determinar la **dirección de salto**:

1. El agente que interrumpe suministra un **vector**, que identifica la dirección del programa que desea que lo atienda (rutina de tratamiento de la interrupción).
2. La unidad de control toma dicha dirección de la **tabla de interrupciones** y la carga en el PC, con lo que comienza a ejecutarse la rutina de tratamiento.

- La tabla de interrupciones la prepara el sistema operativo: cada entrada apunta a una rutina de tratamiento.



Adaptado de Carretero et al., Sistemas operativos: una visión aplicada, cap. 1.

Causas de las interrupciones, retorno y prioridades

Causa de la interrupción	Ejemplo
1. Excepciones de programa por problemas de ejecución	Desbordamiento, operaciones aritméticas no permitidas
2. Interrupciones de reloj	Fin del intervalo de tiempo asignado
3. Interrupciones de E/S	Un dispositivo terminó una transferencia
4. Excepciones del hardware	Error de paridad en la memoria
5. Instrucciones TRAP	Llamada al sistema operativo desde un programa

- **Instrucción RETI** (retorno de interrupción): restituye el registro de estado y el PC, para volver al punto de ejecución donde se generó la interrupción.
- **Prioridades**: hay diferentes señales que solicitan interrupciones, con diferentes prioridades.
- **Inhibición**: algunas señales se pueden inhibir explícitamente (enmascarar), por lo que no serán atendidas; las no enmascarables (NMI) se atienden siempre.

Las excepciones y las TRAP son **síncronas** (las provoca la propia instrucción); las de reloj, E/S y hardware son **asíncronas** respecto al programa en ejecución.



Reloj

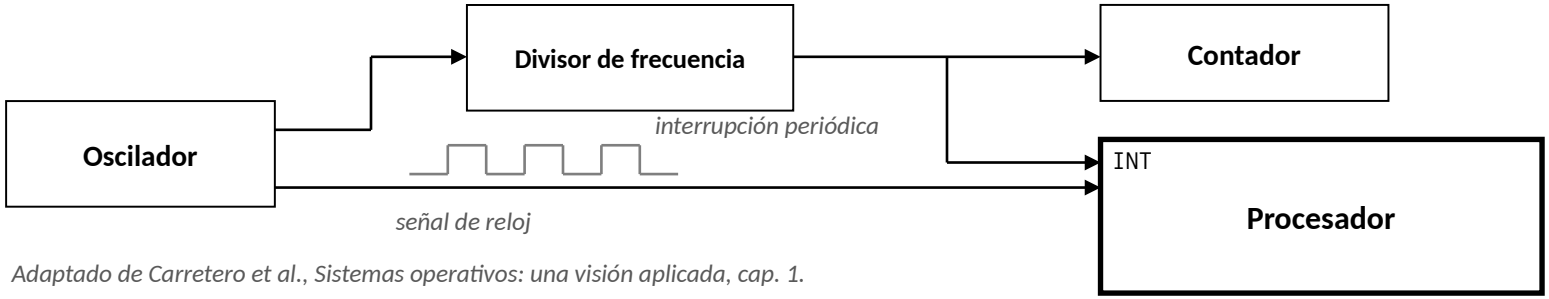
Tres visiones del reloj

1. Oscilador
Gobierna las fases de las instrucciones de máquina. Su frecuencia (p. ej. 1000 MHz) marca el ritmo del procesador.

2. Interrupción periódica
La señal del oscilador se divide para generar una interrupción cada cierto periodo de tiempo. Objetivo: que el sistema operativo pase a ejecutar de forma sistemática cada cierto intervalo, evitando que un programa acapare los recursos del sistema.

3. Contador de fecha y hora
Contabiliza unidades de tiempo (p. ej. segundos) desde un instante dado (p. ej. en UNIX, las 0 h del 1 de enero de 1970). Esta cuenta la puede llevar un hardware especial con batería o el sistema operativo.

Esquema general con las tres visiones del reloj



Adaptado de Carretero et al., Sistemas operativos: una visión aplicada, cap. 1.



Segmentación y arquitecturas RISC y CISC

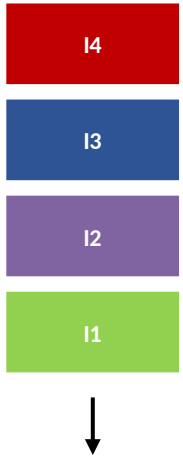
Segmentación, anticipación y paralelismo

- **Segmentación** (pipeline): descomponer la ejecución de cada instrucción en varias etapas, para poder empezar a procesar una instrucción diferente en cada una de ellas y trabajar con varias a la vez.
- **Anticipación**: búsqueda «anticipada» de instrucciones para superponer las fases de búsqueda, decodificación y ejecución.
- **Paralelismo funcional**: tareas que se pueden ejecutar al mismo tiempo.
  - **Espacial**: replicación de hardware.
  - **Temporal**: solapamiento de operaciones funcionales en el tiempo (segmentación).
- **Paralelismo de datos**: se ejecuta una misma instrucción sobre un conjunto de datos.

Segmentación: cauce de cuatro etapas

	ciclo 1	ciclo 2	ciclo 3	ciclo 4	ciclo 5	ciclo 6	ciclo 7
Etapa 1: leer (fetch)	I1	I2	I3	I4	—	—	—
Etapa 2: decodificar	—	I1	I2	I3	I4	—	—
Etapa 3: ejecutar	—	—	I1	I2	I3	I4	—
Etapa 4: escribir resultado	—	—	—	I1	I2	I3	I4

Instrucciones en espera



entran al cauce una por ciclo

- En cada ciclo, cada etapa trabaja sobre una instrucción distinta; con el cauce lleno se termina **una instrucción por ciclo**.
- Sin segmentación, 4 instrucciones de 4 etapas necesitan 16 ciclos; con el cauce, 7.

Arquitecturas RISC y CISC

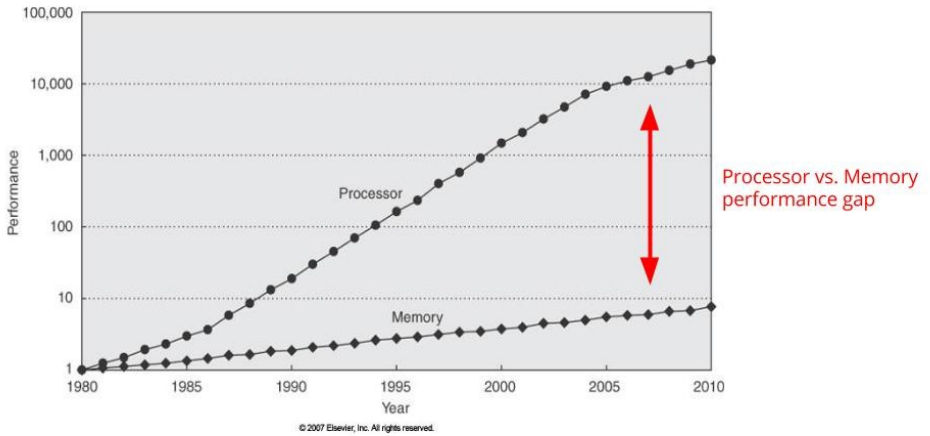
Aspecto	RISC	CISC
Juego de instrucciones	Número limitado de instrucciones simples.	Muchas instrucciones complejas.
Formato	Instrucciones de longitud fija (32 bits) y codificación con campos fijos.	Instrucciones de longitud variable.
Operandos	Operaciones registro a registro (arquitectura load/store).	Operaciones tipo memoria a registro.
Registros	Alto número de registros de propósito general (32).	Pocos registros «generales/implícitos» (8).
Opcionalmente	• Arquitectura de 64 bits. • 3 direccionamientos: registro, inmediato, desplazamiento. • Un único modo de direccionamiento para load/store: base + desplazamiento. • Instrucciones aritméticas tipo reg-reg de 3 direcciones.	• Arquitectura de 32/64 bits. • 2-3 direccionamientos. • Varios modos de direccionamiento en las instrucciones load/store.

RISC: Reduced Instruction Set Computer · CISC: Complex Instruction Set Computer.



Procesador, memoria y almacenamiento

La brecha entre el procesador y la memoria



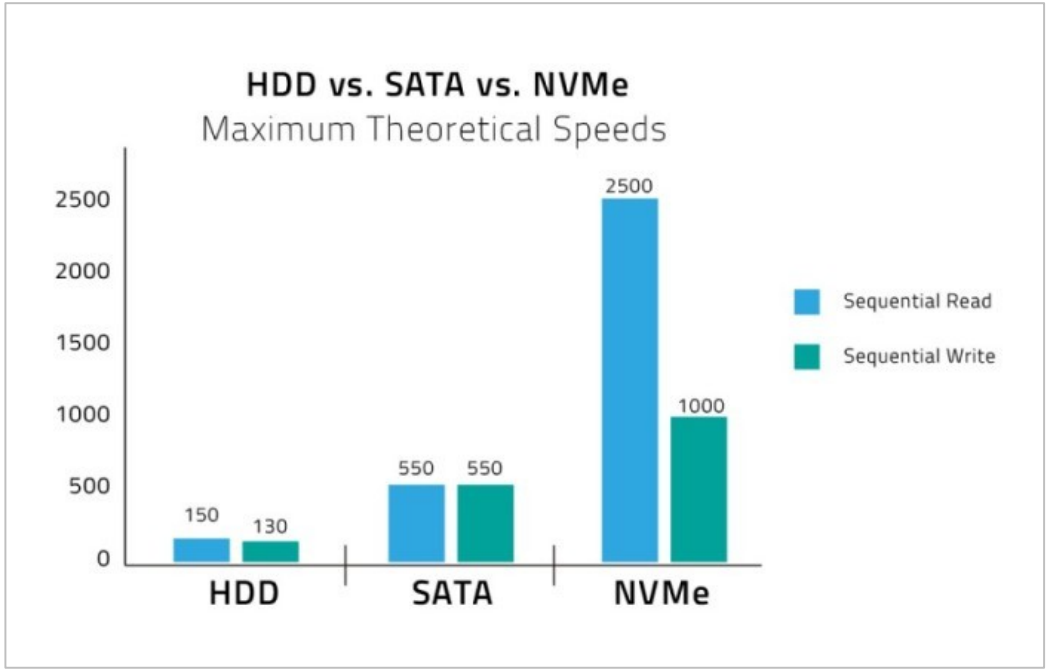
Memory speed lags behind CPU speed

Rendimiento relativo (1980 = 1) del procesador y de la memoria: la memoria mejora mucho más despacio que la CPU. Fuente: Hennessy y Patterson, Computer Architecture: A Quantitative Approach, 4.ª ed., Elsevier, 2007.

Operación	Tiempo
Ejecutar una instrucción típica	1 ns
Leer de la caché L1	0,5 ns
Predicción de salto fallida	5 ns
Leer de la caché L2	7 ns
Bloquear / desbloquear un mutex	25 ns
Leer de la memoria principal	100 ns
Enviar 2 KB por una red de 1 Gbps	20 000 ns
Leer 1 MB secuencial de memoria	250 000 ns
Buscar una posición en disco (seek)	8 000 000 ns
Leer 1 MB secuencial de disco	20 000 000 ns
Paquete EE. UU. – Europa, ida y vuelta	150 000 000 ns

Latencias aproximadas («Latency numbers every programmer should know», J. Dean y P. Norvig).

Velocidad del almacenamiento: HDD, SSD SATA y SSD NVMe

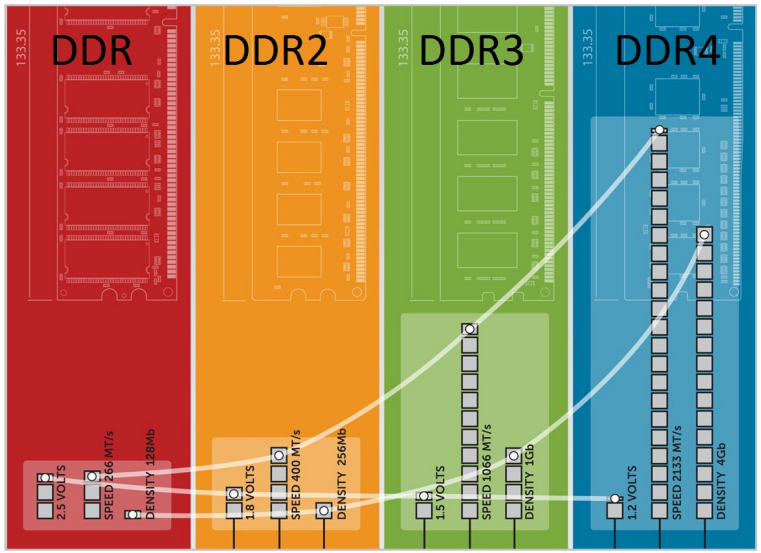


Velocidades máximas teóricas (MB/s). Barras azules: lectura secuencial; barras verdes: escritura secuencial. Fuente: material del curso.

Medio	Lectura	Escritura
Disco duro (HDD)	150 MB/s	130 MB/s
SSD SATA	550 MB/s	550 MB/s
SSD NVMe	2500 MB/s	1000 MB/s

- El disco mecánico está limitado por el movimiento de los cabezales y el giro de los platos.
- El SSD SATA elimina la mecánica, pero la interfaz SATA III lo limita a unos 600 MB/s.
- El SSD NVMe usa el bus PCI Express y multiplica la velocidad de transferencia.

Generaciones de memoria DDR SDRAM

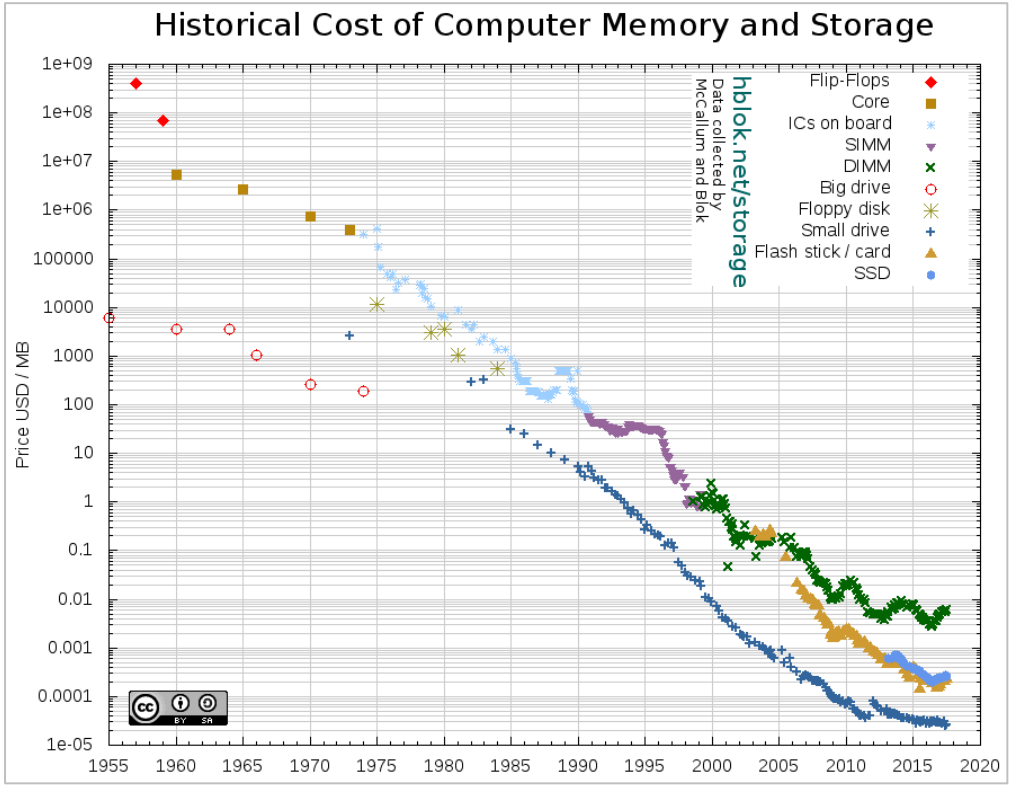


Tensión, velocidad y densidad de cada generación. Fuente: material del curso.

Estándar	Reloj interno (MHz)	Reloj del bus (MHz)	Prefetch	Tasa de datos (MT/s)	Transferencia (GB/s)	Tensión (V)
SDRAM	100 – 166	100 – 166	1n	100 – 166	0,8 – 1,3	3,3
DDR	133 – 200	133 – 200	2n	266 – 400	2,1 – 3,2	2,5 / 2,6
DDR2	133 – 200	266 – 400	4n	533 – 800	4,2 – 6,4	1,8
DDR3	133 – 200	533 – 800	8n	1066 – 1600	8,5 – 12,8	1,35 / 1,5
DDR4	133 – 200	1066 – 1600	8n	2133 – 3200	17,1 – 25,6	1,2

- Con el mismo reloj interno de las celdas (133 – 200 MHz), de SDRAM a DDR3 cada generación duplica el **prefetch** y la tasa de datos; DDR4 mantiene 8n y duplica el reloj del bus.
- La tensión baja de 3,3 V a 1,2 V: menos consumo y menos calor por módulo.
- MT/s: millones de transferencias por segundo; GB/s: ancho de banda de un módulo de 64 bits.

Coste histórico de la memoria y el almacenamiento



- Precio en dólares por MB (escala logarítmica) de cada tecnología: biestables, núcleos de ferrita, circuitos integrados, SIMM y DIMM, discos, disquetes, memorias flash y SSD.
- De cientos de millones de dólares por MB (1957) a diezmilésimas de dólar por MB (2017).
- La memoria (DIMM) sigue siendo decenas de veces más cara por MB que el disco o el SSD: de ahí la **jerarquía de memoria** (próximo tema).

Fuente: J. C. McCallum y H. Blok, «Historical cost of computer memory and storage», hblok.net/storage (CC BY-SA).

Resumen

- Computador de **programa almacenado** (Von Neumann): un único circuito ejecuta la codificación de cada tarea; instrucciones y datos comparten la memoria.
- La unidad de control repite sin fin el ciclo **leer – decodificar – ejecutar** con PC, IR, MAR, MDR y el acumulador.
- El **modelo de programación**: elementos de almacenamiento, juego de instrucciones, secuencia de funcionamiento y niveles de ejecución (modo usuario y modo núcleo).
- Las **interrupciones** avisan al procesador: el ciclo de aceptación guarda PC y estado, eleva el nivel y salta a la rutina de tratamiento (vector); RETI restaura el contexto.
- El **reloj**: oscilador, interrupción periódica (el SO retoma el control) y contador de fecha y hora.
- Segmentación y paralelismo aumentan el rendimiento; RISC y CISC son dos filosofías de juego de instrucciones. La brecha procesador–memoria motiva la jerarquía de memoria.

Referencias y lecturas

- Carretero, García, de Miguel y Pérez. *Sistemas operativos: una visión aplicada*, 2.ª ed., McGraw-Hill, 2007. Capítulo 1: modelo de Von Neumann, modelos de programación, interrupciones y reloj.
- Stallings. *Organización y arquitectura de computadores*, 7.ª ed., Pearson Prentice Hall, 2006. Ciclo de instrucción, interrupciones, segmentación y RISC/CISC.
- Silberschatz, Galvin y Gagne. *Operating System Concepts*, 10.ª ed., Wiley, 2018. Capítulo 1: ciclo de E/S regido por interrupciones.
- Hennessy y Patterson. *Computer Architecture: A Quantitative Approach*, 4.ª ed., Morgan Kaufmann / Elsevier, 2007. Gráfica de la brecha procesador–memoria.
- Dean y Norvig. «Latency numbers every programmer should know» (tabla de latencias).
- McCallum y Blok. «Historical cost of computer memory and storage», hblok.net/storage (CC BY-SA).